Program Structure

Overview of how `fem2d` works, from the user's input to the visualization (as of 30 Oct 2026).

0. Starting point

Currently only f and Dirichlet (g) are supported, the rest is planned.

The user has

- a surface (domain Ω)
- a source $f(x)$ (e.g. "how strongly it is heated")
- a material coefficient $k(x)$
- the boundary conditions: which boundary section has which type and with which function it holds (e.g. for Dirichlet, u is prescribed on the boundary as $u = g$, or for Neumann, the flux through the boundary, e.g. $-k\,\nabla u \cdot n = j$)

The user wants to determine the scalar field that develops on their surface from these conditions, e.g. the temperature distribution.

1. Separate instance: Gmsh

The user uses Gmsh to create a mesh of triangles for their surface.

- The input is given in a `.geo` file. It contains
  - the corner points
  - the lines between them
  - the surfaces they enclose
  - the desired triangle size `h`
  - named groups like "Physical Curve" and "Physical Surface", which mark which boundary later gets which condition and which subdomain consists of which material
- The `.geo` is Gmsh's scripting language as plain text, which describes the geometry relatively simply; from it, Gmsh generates the decomposition into triangles. Example `meshes/square.geo`:

```
// Unit square. Generate the mesh with:
//   gmsh -2 -format msh2 meshes/square.geo -o meshes/square.msh
h = 0.1;
Point(1) = {0, 0, 0, h};
Point(2) = {1, 0, 0, h};
Point(3) = {1, 1, 0, h};
Point(4) = {0, 1, 0, h};
Line(1) = {1, 2};
Line(2) = {2, 3};
Line(3) = {3, 4};
Line(4) = {4, 1};
Curve Loop(1) = {1, 2, 3, 4};
Plane Surface(1) = {1};
Physical Curve("boundary") = {1, 2, 3, 4};
Physical Surface("domain") = {1};
```

- For more complicated geometries or automation, one uses CAD programs like FreeCAD, SolidWorks or Fusion
  - exported as STEP, the standard format for exact geometry, the counterpart to `.obj` for render meshes
  - Gmsh can read STEP directly and generate an FEM-suitable mesh from it
- Gmsh produces a `.msh` file, which consists of two things:
  - Block `$Nodes`: lists all nodes with number and coordinates (x, y, z), where $z = 0$ here because of 2D
  - Block `$Elements`: lists the elements; each one has
    - a type
      - type 1 are the line segments on the boundary with two node numbers
      - type 2 are the triangles with three node numbers
    - a few tags, e.g. which Physical group an element belongs to (label, boundary, material)
    - the numbers of its nodes
- Since Gmsh writes format 4 by default and the reader needs format 2, the `README.md` asks the user to add `-format msh2` to the Gmsh call.

2. Reading the `.msh`

Goal: The `.msh` is translated into a data structure the program can compute with, our tables.

`struct Mesh`

Defined in `include/fem/mesh.hh`, currently as

```
struct Mesh {
  std::vector<std::array<double, 2>> points;          // N x 2 coordinates
  std::vector<std::array<std::size_t, 3>> triangles;  // M x 3 corner point indices
  std::vector<bool> on_boundary;                      // size N, true for boundary points
};
```

- Outer container: a `std::vector` each, because the number of nodes and triangles is only known when reading and has to grow
- Inner container: a `std::array`, because every point has exactly two coordinates and every triangle exactly three corners

The tables (example from `tests/test_assembly.cc`):

- `points`: every node gets a number starting at 0 and its coordinates:

$$0 : (0, 0), \quad 1 : (1, 0), \quad 2 : (1, 1), \quad 3 : (0, 1), \quad 4 : (0.5, 0.5)$$

- `triangles`: the element table; each triangle stores no coordinates, only the numbers of its three corners, following the global/local principle:

$$T_0 = (0, 1, 4), \quad T_1 = (1, 2, 4), \quad T_2 = (2, 3, 4), \quad T_3 = (3, 0, 4)$$

  - The corners are called 0, 1, 2 locally
  - The entry `triangles[t][i]` says which global node that is → the $g(k, i)$ from the slides of the first meeting
  - Advantage: repeated nodes are stored only once in `points`, which connects the triangles and makes u_h continuous across the edges

- `on_boundary`:
  - stores one true/false per node, depending on whether the node lies on the boundary
  - is determined by the reader from the type 1 line segments of the `.msh`

`read_gmsh_msh2`

Reads the file. Located in `include/fem/mesh_reader.hh`, currently defined as

```
inline Mesh read_gmsh_msh2(const std::string& filename)
```

- `inline` defines `read_gmsh_msh2` as an inline function, so the linker does not complain when the `.hh` file is included in several files (like `main.cc` and a test file at the same time); `inline` tells the linker that the definition may appear in several files because the copies are identical
- takes as parameter a string with the path to the `.msh` and returns a fully filled `Mesh` object

How it works:

1. Open the file and read it word by word
2. When you hit `$Nodes`, fill `points`
3. When you hit `$Elements`, sort them by type: triangles go into `triangles`, boundary segments into a helper list
4. Mark `on_boundary` using this temporary list

The function is used in `src/main.cc` via

```
fem::Mesh mesh = fem::read_gmsh_msh2(argv[1]);
```

3. Assembly: building the large linear system $Au = b$ (still to be implemented)

- File: `include/fem/assembly.hh`, function `assemble_poisson` (still empty)
- Approach: `assemble_poisson` gets the `Mesh` and the user functions (`Problem`) and returns a `LinearSystem`, i.e. the matrix `A` as `hdnum::SparseMatrix` and the vector `b`

How it works:

1. Create an empty builder for an $N \times N$ matrix (N = number of nodes) and a zero vector `b` of length N.
2. The loop runs over all triangles in `triangles`. For each triangle, get the corner coordinates from `points` via the three global numbers and compute two things from them:
   1. the local stiffness matrix with the formula

$$K_{ij}^T = \int_T \nabla\varphi_{g(t,j)} \cdot \nabla\varphi_{g(t,i)}\, dx = \frac{\beta_i\beta_j + \gamma_i\gamma_j}{4A}, \qquad i, j \in \{0, 1, 2\}$$

from `docs/math.md`, where A is the area of the triangle and β and γ are the coordinate differences of the corners (called b, c in `docs/math.md`) → this gives 9 entries per triangle

2. the local load vector with three entries, which says how much of the source f goes to each of the three corners (simplest approximation: each corner gets one third, $f \cdot A/3$)

3. Insert:

   - Add the local entry (i, j) to the global position (`triangles`$[t][i]$, `triangles`$[t][j]$) of the large matrix (because a node belongs to several triangles and all of them add their contribution):

$$A_{kl} = \int_\Omega \nabla\varphi_l \cdot \nabla\varphi_k \, dx = \sum_{T \ni k,\, l} K^T_{\text{local}(k),\,\text{local}(l)}$$

     → if no triangle contains both nodes, $A_{kl} = 0$

   - Add entry i of the local load vector b^T

$$b_i^T = \int_T f \, \varphi_{g(t,i)} \, dx \approx f \cdot \frac{A}{3}, \qquad i \in \{0, 1, 2\}$$

     to `b[triangles[t][i]]`, the global entry as the sum of the contributions of all triangles around node k:

$$b_k = \int_\Omega f \, \varphi_k \, dx = \sum_{T \ni k} b^T_{\text{local}(k)}$$

4. Apply boundary conditions: for every node with `on_boundary[i] == true`, replace its row by $u_i = g(x_i)$: a 1 on the diagonal, zeros elsewhere, and $g(x_i)$ in `b`.

5. Build the matrix: `builder.build()` turns the collected entries into the final sparse matrix.

4. Solving $Au = b$

Already implemented in `src/main.cc`:

```
auto LU = hdnum::sparseLUDecomposition(sys.A);
hdnum::Vector<double> u = hdnum::sparseLUSolver(LU, sys.b);
```

How it works:

1. Decompose the matrix A into a lower and an upper triangular matrix, $A = LU$
2. Solve the system by forward and backward substitution, $Ly = b$ and $Uu = y$ (Gaussian elimination for sparse matrices)
3. Result: a vector `u` with one value per node as the approximate solution (e.g. for heat, the temperature at every grid point)

5. Output

- `write_vtk` from `include/fem/vtk_writer.hh` writes `points`, `triangles` and `u` into a `.vtk` file
- the user opens it in ParaView